

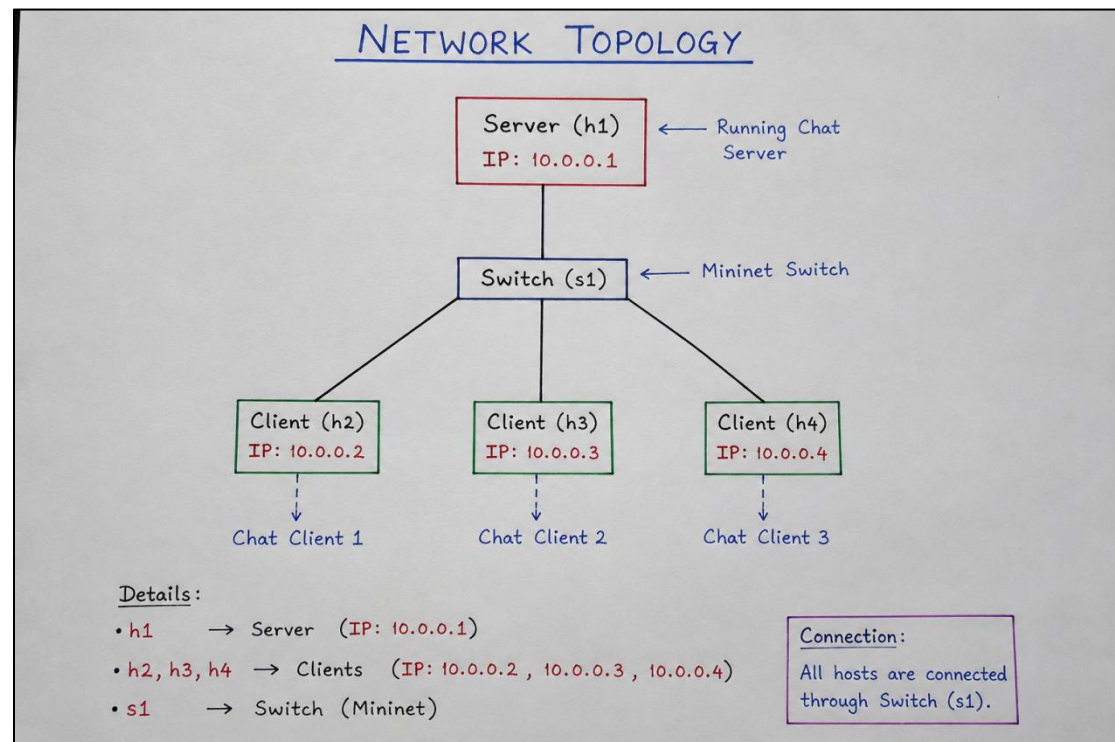
Assignment Report on Multi-Client Chat Server using TCP Sockets

Submitted By	
Student Name:	Shahan Uz Zaman
Roll Number:	CS-BTC24-23
Course / Subject Name	Summer internship on Cyber Security and Secure App Development

1. Objective

The objective of this assignment is to develop a multi-client chat application using TCP socket programming in Python. The application allows multiple clients to communicate simultaneously through a central server. The server manages client connections using threads, broadcasts messages to connected users, logs chat activities, measures communication performance, and verifies TCP communication using Wireshark. These tasks correspond to the assignment requirements.

2. Network Topology



The topology consists of one server and three clients created using Mininet.

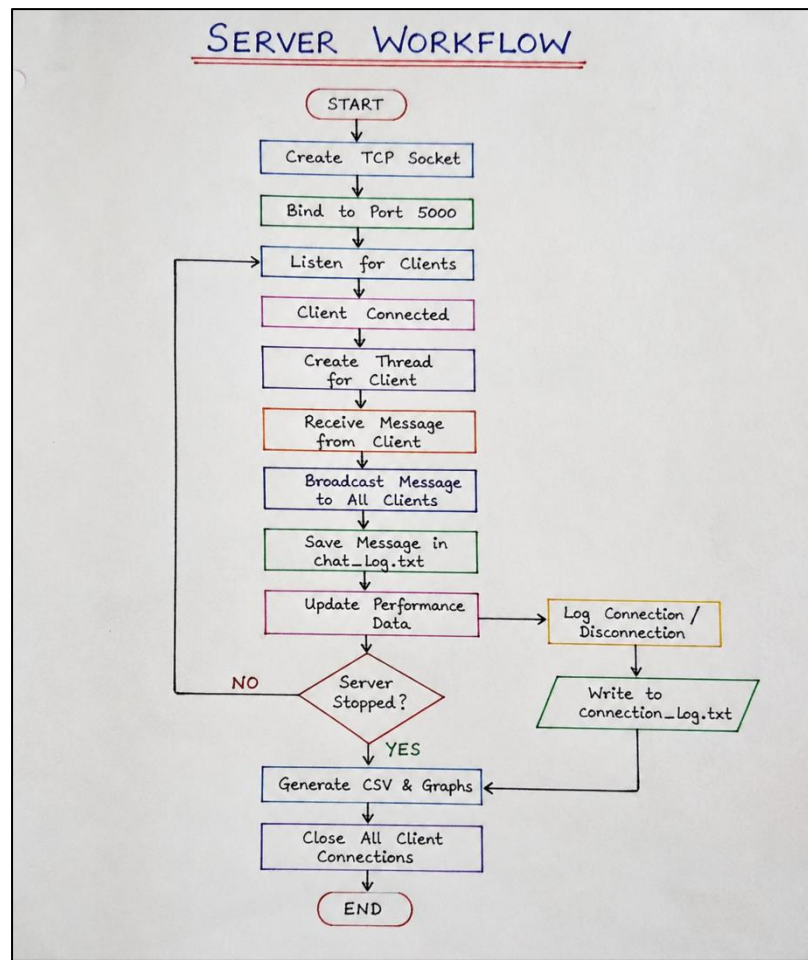
3. Program Design

Server

The server performs the following tasks:

- Creates a TCP socket.
- Listens on port 5000.
- Accepts multiple client connections.
- Creates one thread per client.
- Receives messages.
- Broadcasts messages to all connected clients.
- Logs connection and disconnection events.
- Stores chat messages in chat_log.txt.
- Measures delivery delay and throughput.
- Generates graphs when the server is terminated.
- Closes all client connections gracefully.

Server Workflow



Client

Each client:

- Connects to the server.
- Sends its username.
- Sends chat messages.
- Receives broadcast messages.
- Disconnects by typing exit.

4. Implementation

Task 1 – Multi-Client Chat Server

Implemented using:

- TCP sockets
- Python threading
- Active client list
- Connection logging

Server log example:

- 10:10:12 CONNECTED shahan 10.0.0.2
- 10:15:20 DISCONNECTED shahan 10.0.0.2

This satisfies the requirement to accept multiple clients and maintain connection logs.

Task 2 – User Registration and Broadcast Messaging

Each client registers using a username after connecting to the server.

Example:

- Enter Username: shahan

Broadcast message:

- [shahan] Hello Everyone

All messages are stored in:

- chat_log.txt

This implements the assignment requirement for username registration, broadcasting, and chat logging.

Task 3 – Performance Measurement**The server measures:**

- Number of clients
- Total messages
- Average delivery delay
- Throughput

The results are stored in:

- performance_results.csv

The server automatically generates graphs when it is stopped using the exit command.

5. Wireshark Analysis

TCP packets were captured using Wireshark with the filter:

- tcp.port == 5000

The capture verifies:

- TCP Three-Way Handshake
- Username transmission
- Broadcast messages
- TCP acknowledgements

Your capture includes:

- Client SYN to server on TCP port 5000
- Server SYN-ACK response
- Client ACK completing the connection
- Username transmission (shahan)
- Broadcast packet (** shahan joined the chat **)
- Broadcast chat message ([shahan] m1)

6. Experimental Results

The following files were generated:

- connection_log.txt
- chat_log.txt
- performance_results.csv

Graphs:

- clients_vs_delay.png
- clients_vs_throughput.png

Wireshark screenshots:

- tcp_handshake.png
- client_username.png
- broadcast_join.png
- chat_message.png

7. Advantages

- Supports multiple clients simultaneously.
- Uses TCP for reliable communication.
- Thread-based concurrent processing.
- Automatic logging of chat activity.
- Automatic performance analysis.
- Graph generation for result visualization.
- Easy to extend with additional features.

8. Limitations

- Runs on a local Mininet network.
- Messages are transmitted as plain text.
- No authentication or encryption.
- Performance decreases as the number of clients increases significantly.
- Graphs depend on the collected experimental data.

9. Conclusion

This assignment successfully implemented a multi-client chat server using TCP socket programming in Python. The server handled concurrent client connections using threads, supported user registration and message broadcasting, maintained connection and chat logs, and measured communication performance. Wireshark analysis verified TCP communication between the clients and server, and the generated CSV file and graphs provided insight into delivery delay and throughput. Overall, the project demonstrated practical concepts of TCP networking, concurrent programming, packet analysis, and performance evaluation as required by the assignment.